# ⚙️ AWS Infrastructure & Automation — Category 10: Complete Interview Guide

> **Target Audience:** DevOps / SRE / Platform / Cloud Engineers
> **Difficulty:** Beginner → Advanced
> **Topics Covered:** SSM Session Manager, Run Command, Patch Manager, State Manager, Terraform on AWS, CDK, SCPs, Control Tower, RAM, Tag Policies

---

## 📋 Table of Contents


1. 10.1 Systems Manager — Session Manager, Parameter Store, Run Command
2. 10.2 SSM — Patch Manager, Inventory, State Manager
3. 10.3 Terraform on AWS — State Management with S3 + DynamoDB
4. 10.4 AWS CDK — Constructs, Stacks, Bootstrapping
5. 10.5 Service Control Policies — Guardrails at Org Level
6. 10.6 AWS Control Tower — Landing Zones, Guardrails
7. 10.7 Resource Access Manager (RAM)
8. 10.8 Tag Policies & Cost Allocation


---

# 10.1 Systems Manager — Session Manager, Parameter Store, Run Command

---

## 🟢 What It Is in Simple Terms

AWS Systems Manager (SSM) is AWS's operations management platform for EC2 instances and on-premises servers. It lets you manage fleets of servers without SSH, without opening inbound ports, without storing SSH keys — through a single unified control plane that integrates with IAM, CloudTrail, and CloudWatch.

---

## 🔍 Why It Exists / What Problem It Solves

```
Traditional SSH management problems:
├── Bastion hosts: extra cost, extra attack surface, single point of
failure
├── SSH key distribution: keys get lost, shared, forgotten, never rotated
├── Inbound port 22: security risk, firewall complexity, audit concerns
├── No central audit trail: who ran what command on which server?
└── Human error: wrong server, wrong command, no undo

SSM solves all of this:
```

```
├── No SSH keys — IAM controls who can do what, on which instances
├── No inbound ports — SSM agent communicates OUTBOUND to SSM endpoints
├── Full audit trail — every session, every command in CloudTrail
├── Centralized control — manage 10 or 10,000 instances from one place
└── Works on EC2 AND on-premises servers (with SSM agent installed)
```

---

## ⚙️ SSM Agent and Connectivity

```
SSM Agent Architecture:

EC2 Instance                      AWS SSM Service
┌──────────────────┐              ┌──────────────────────┐
│  SSM Agent       │──OUTBOUND──►│  SSM Endpoints       │
│  (pre-installed  │  HTTPS 443   │  ssm.us-east-1.      │
│   on Amazon      │              │  amazonaws.com       │
│   Linux / Win)   │              └──────────────────────┘
└──────────────────┘
        │
        Instance Role must allow:
        ssm:RegisterManagedInstance
        ssm:UpdateInstanceInformation
        ssm:ListAssociations
        ec2messages:GetMessages
        ssmmessages:CreateControlChannel

Minimum IAM policy (attach to EC2 instance role):
AmazonSSMManagedInstanceCore (AWS managed policy)

VPC requirements (if no internet access):
Instance → VPC Endpoint for SSM (com.amazonaws.region.ssm)
Instance → VPC Endpoint for SSMMesages (com.amazonaws.region.ssmmessages)
Instance → VPC Endpoint for EC2Messages (com.amazonaws.region.ec2messages)

No VPC endpoint + no NAT = instance NOT visible in SSM
```

---

## 🧩 Session Manager

```
Session Manager = browser-based or CLI shell access to EC2
                  WITHOUT SSH, WITHOUT bastion hosts, WITHOUT port 22

How it works:
1. User initiates session via Console or CLI
2. SSM service authenticates via IAM
3. SSM establishes encrypted tunnel through existing outbound connection
4. User gets interactive shell (bash/PowerShell)
5. All keystrokes and output logged to CloudTrail + S3 + CloudWatch Logs
```

```
Session audit trail:
├── CloudTrail: who started session, when, from which IP
├── S3: full session transcript (every keystroke + output)
└── CloudWatch Logs: real-time session stream for monitoring/alerting

Starting a session:
# CLI
aws ssm start-session \
  --target i-0abc123def456 \
  --region us-east-1

# Port forwarding (no inbound ports needed!)
aws ssm start-session \
  --target i-0abc123def456 \
  --document-name AWS-StartPortForwardingSession \
  --parameters '{"portNumber":["5432"],"localPortNumber":["15432"]}'
# Now: psql -h localhost -p 15432 → connects to RDS through the instance
# No SG inbound rule needed for port 5432 at all!

# SSH over SSM (standard SSH client, zero SSH port open)
aws ssm start-session \
  --target i-0abc123def456 \
  --document-name AWS-StartSSHSession \
  --parameters 'portNumber=22'
# ~/.ssh/config ProxyCommand = use SSM as proxy
# Still no inbound port 22 on EC2!

IAM policy to allow Session Manager:
{
  "Effect": "Allow",
  "Action": [
    "ssm:StartSession",
    "ssm:TerminateSession",
    "ssm:ResumeSession",
    "ssm:DescribeSessions",
    "ssm:GetConnectionStatus"
  ],
  "Resource": [
    "arn:aws:ec2:*:*:instance/*",
    "arn:aws:ssm:*:*:session/${aws:username}*"
  ],
  "Condition": {
    "StringEquals": {
      "ssm:resourceTag/Environment": "prod"
    }
  }
}
# ABAC: can only start sessions on instances tagged Environment=prod
```

---

## 🧩 Run Command

```
Run Command = execute scripts/commands on one or many instances
              simultaneously, without SSH

Use cases:
├── Deploy application across 500 instances in one command
├── Rotate credentials on all instances at once
├── Install packages, apply configuration changes
├── Run diagnostic scripts during incident response
└── Trigger one-time maintenance tasks on instance fleets

Run Command key features:
├── Target by: instance ID, tag, resource group, or ALL managed instances
├── Concurrent rate control: run on N at a time or N% at a time
├── Error threshold: stop if X% of instances fail
├── Full output captured: per-instance stdout/stderr in S3 or CloudWatch
└── Audit trail: all executions in CloudTrail

Running a shell script on all prod web servers:
aws ssm send-command \
  --document-name "AWS-RunShellScript" \
  --targets '[{"Key":"tag:Role","Values":["web-server"]},
              {"Key":"tag:Environment","Values":["prod"]}]' \
  --parameters '{
    "commands": [
      "#!/bin/bash",
      "cd /app",
      "git pull origin main",
      "systemctl restart myapp",
      "systemctl status myapp"
    ]
  }' \
  --timeout-seconds 300 \
  --max-concurrency "25%" \
  --max-errors "10%" \
  --output-s3-bucket-name my-ssm-output-bucket \
  --output-s3-key-prefix "run-command/deploy/$(date +%Y-%m-%d)"

# Check execution status
aws ssm list-command-invocations \
  --command-id abc-123-def \
  --details

# View output for specific instance
aws ssm get-command-invocation \
  --command-id abc-123-def \
  --instance-id i-0abc123
```

```
AWS-Provided Documents (pre-built SSM documents):
├── AWS-RunShellScript:         run bash on Linux
├── AWS-RunPowerShellScript:    run PowerShell on Windows
├── AWS-RunAnsiblePlaybook:     run Ansible playbook
```

```
├── AWS-ApplyAnsiblePlaybooks:  apply Ansible from S3
├── AWS-InstallWindowsUpdates:  Windows update management
├── AWS-ConfigureAWSPackage:    install AWS packages (CloudWatch Agent,
etc.)
├── AWS-UpdateSSMAgent:         update SSM agent itself
└── AWS-StartPortForwardingSession: port forwarding tunnel

Custom SSM Documents (SSM documents = JSON/YAML automation definitions):
{
  "schemaVersion": "2.2",
  "description": "Install and configure Nginx",
  "parameters": {
    "serverPort": {
      "type": "String",
      "default": "80",
      "description": "Port for Nginx to listen on"
    }
  },
  "mainSteps": [{
    "action": "aws:runShellScript",
    "name": "installNginx",
    "inputs": {
      "runCommand": [
        "yum install -y nginx",
        "sed -i 's/listen 80/listen {{serverPort}}/'
/etc/nginx/nginx.conf",
        "systemctl enable nginx && systemctl start nginx"
      ]
    }
  }]
}
```

---

## 🧩 Parameter Store (SSM)

```
(Covered in depth in Category 8 Security — quick recap here)

Parameter Store in the context of SSM automation:

Common SSM automation uses for Parameter Store:
├── Store CloudWatch Agent configuration → fetch on instance startup
├── Store application configs → fetch in UserData/launch scripts
├── Store fleet-wide settings → SSM Automation reads during patching
└── Store AMI IDs → automation references latest AMI by name not ID

# Fetch Parameter Store value in EC2 UserData (launch time)
#!/bin/bash
DB_HOST=$(aws ssm get-parameter \
  --name "/prod/myapp/db/host" \
  --query "Parameter.Value" \
  --output text)
```

```
DB_PASS=$(aws ssm get-parameter \
  --name "/prod/myapp/db/password" \
  --with-decryption \
  --query "Parameter.Value" \
  --output text)

echo "DB_HOST=$DB_HOST" >> /etc/myapp/config.env
echo "DB_PASS=$DB_PASS" >> /etc/myapp/config.env
systemctl start myapp

# Reference Parameter Store in CloudFormation / CDK
# {{resolve:ssm:/prod/myapp/db/host}} resolves at deploy time
```

---

## 💬 Short Crisp Interview Answer

*"SSM is AWS's unified server management platform. Session Manager provides SSH-equivalent shell access without opening any inbound ports, without SSH keys, and with full audit trails to CloudTrail and S3 — every keystroke logged. Run Command executes scripts on one or thousands of instances simultaneously, with concurrent rate control and error thresholds so a bad deployment doesn't roll out to all servers. The SSM Agent on instances communicates outbound only over HTTPS to SSM endpoints, requiring only the AmazonSSMManagedInstanceCore IAM policy on the instance role. In private subnets without internet access, three VPC endpoints are required: ssm, ssmmessages, and ec2messages. Parameter Store integrates with SSM automation to dynamically fetch configuration during instance bootstrap."*

---

## ⚠ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| SSM Agent not on old AMIs | Pre-2017 AMIs may not have SSM agent pre-installed — must install manually |
| VPC endpoints required in private subnets | No NAT + no VPC endpoints = instance invisible to SSM despite having correct IAM role |
| Instance role is mandatory | EC2 without instance role = cannot connect to SSM regardless of agent version |
| Session logging not default | S3/CloudWatch session logging must be explicitly configured in Session Manager preferences |
| Run Command output size limit | Console shows truncated output — use S3 or CloudWatch for full output |
| On-premises servers | Need SSM activation code + ID to register on-premises servers as managed instances |

---

# 10.2 SSM — Patch Manager, Inventory, State Manager

---

## 🟢 What It Is in Simple Terms

Three SSM capabilities for fleet-scale operations. Patch Manager automates OS and application patching across your entire instance fleet. Inventory collects software, configuration, and hardware data from every instance. State Manager ensures instances stay in a defined configuration state over time.

---

## 🧩 Patch Manager

```
Patch Manager automates:
├── Scanning instances for missing patches (compliance assessment)
├── Installing missing patches (patching operation)
├── Reporting patch compliance status
└── Scheduling recurring patching maintenance windows

Key concepts:

Patch Baseline:
├── Defines WHICH patches to approve/reject
├── Rules: auto-approve patches by severity, classification, age
├── Exceptions: explicitly approve or reject specific patches
└── AWS provides default baselines per OS; you can create custom ones

Example: Custom patch baseline for Amazon Linux 2
{
  "Name": "prod-amazon-linux2-baseline",
  "OperatingSystem": "AMAZON_LINUX_2",
  "ApprovalRules": {
    "PatchRules": [{
      "PatchFilterGroup": {
        "PatchFilters": [
          {"Key": "PRODUCT", "Values": ["AmazonLinux2"]},
          {"Key": "CLASSIFICATION", "Values": ["Security","Bugfix"]},
          {"Key": "SEVERITY", "Values": ["Critical","Important"]}
        ]
      },
      "ApproveAfterDays": 7,   ← auto-approve after 7 days in repo
      "ComplianceLevel": "CRITICAL"
    }]
  },
  "RejectedPatches": ["kernel*"],  ← never auto-patch kernel
  "ApprovedPatches": ["amazon-ssm-agent"]  ← always install SSM agent
updates
}

Patch Groups:
├── Tag instances with "Patch Group" = "prod-web" or "dev-servers"
├── Associate a patch baseline with a patch group
```

```
└── Run patching on specific groups without affecting others

Maintenance Windows:
├── Define safe patching windows (e.g., Sunday 2 AM – 4 AM EST)
├── Tasks: Run Command with AWS-RunPatchBaseline document
├── Rate control: patch N instances at a time (rolling patches)
└── Error threshold: stop if N% fail (prevent widespread outage)
```

```
# Create maintenance window
aws ssm create-maintenance-window \
  --name "prod-weekly-patching" \
  --schedule "cron(0 2 ? * SUN *)" \
  --schedule-timezone "US/Eastern" \
  --duration 2 \
  --cutoff 1 \
  --allow-unassociated-targets false

# Register targets (all instances in patch group "prod-web")
aws ssm register-target-with-maintenance-window \
  --window-id mw-abc123 \
  --resource-type INSTANCE \
  --targets '[{"Key":"tag:PatchGroup","Values":["prod-web"]}]'

# Register patching task
aws ssm register-task-with-maintenance-window \
  --window-id mw-abc123 \
  --task-arn arn:aws:ssm:::document/AWS-RunPatchBaseline \
  --task-type RUN_COMMAND \
  --targets '[{"Key":"WindowTargetIds","Values":["target-abc"]}]' \
  --max-concurrency "25%" \
  --max-errors "10%" \
  --task-invocation-parameters '{
    "RunCommand": {
      "Parameters": {"Operation": ["Install"]},
      "OutputS3BucketName": "my-patch-logs",
      "CloudWatchOutputConfig": {
        "CloudWatchLogGroupName": "/ssm/patching",
        "CloudWatchOutputEnabled": true
      }
    }
  }'

# Check patch compliance across fleet
aws ssm describe-instance-patch-states \
  --instance-ids i-0abc123 i-0def456
# Returns: InstalledCount, MissingCount, FailedCount, ComplianceStatus
```

```
Patch compliance reporting:
├── ComplianceStatus: COMPLIANT / NON_COMPLIANT
```

```
├── MissingCount: patches approved but not installed
├── FailedCount: patches attempted but failed
└── InstalledRejectedCount: patches installed that baseline rejects

⚠ Patch Manager does NOT reboot instances automatically by default.
   Set RebootOption: RebootIfNeeded in task parameters.
   Without reboot, kernel patches require manual reboot to take effect.
```

---

## 🧩 Inventory

```
SSM Inventory = collect software and configuration data from instances

What Inventory collects:
├── Applications:      installed software, version, publisher
├── AWS components:    AWS-specific software (SSM Agent version, etc.)
├── Network config:    IP addresses, MAC addresses, DNS
├── Windows updates:   installed Windows patches
├── Instance details:  CPU, memory, OS version, instance type
├── Services:          running services and status
├── Files:             specific files you configure it to track
└── Custom inventory:  your own data (app version, config hash, etc.)

Setting up Inventory collection:
# Create State Manager association to collect inventory
aws ssm create-association \
  --name "AWS-GatherSoftwareInventory" \
  --targets '[{"Key":"tag:Environment","Values":["prod"]}]' \
  --schedule-expression "rate(30 minutes)" \
  --parameters '{
    "applications":     ["Enabled"],
    "awsComponents":    ["Enabled"],
    "networkConfig":    ["Enabled"],
    "instanceDetailedInformation": ["Enabled"],
    "services":         ["Enabled"],
    "windowsUpdates":   ["Enabled"]
  }'

Querying Inventory data:
# List all instances running a specific software version
aws ssm list-inventory-entries \
  --instance-id i-0abc123 \
  --type-name "AWS:Application" \
  --filters '[{"Key":"Name","Values":["nginx"]}]'

# Use Resource Data Sync to ship inventory to S3 for Athena queries
aws ssm create-resource-data-sync \
  --sync-name "inventory-to-s3" \
  --s3-destination '{
    "BucketName": "my-ssm-inventory",
    "Prefix": "inventory/",
```

```
    "SyncFormat": "JsonSerDe",
    "Region": "us-east-1"
  }'
# Then query with Athena: which instances run nginx < 1.24?
# SELECT * FROM inventory WHERE application_name = 'nginx' AND version <
'1.24'

Custom inventory:
# Push custom data to SSM Inventory (e.g., app version, config hash)
aws ssm put-inventory \
  --instance-id i-0abc123 \
  --items '[{
    "TypeName": "Custom:AppInfo",
    "SchemaVersion": "1.0",
    "CaptureTime": "2024-01-15T14:30:00Z",
    "Content": [{
      "AppVersion":    "2.4.1",
      "ConfigHash":    "sha256:abc123",
      "DeployedBy":    "jenkins-pipeline-456",
      "DeployedAt":    "2024-01-15T10:00:00Z"
    }]
  }]'
```

---

## 🧩 State Manager

```
State Manager = ensures instances remain in a desired configuration state
                over time (not just one-time — continuously enforced)

State Manager vs Run Command:
├── Run Command:    one-time, fire-and-forget command execution
└── State Manager: ongoing, scheduled, drift-detection and remediation
    "Instance must ALWAYS have CloudWatch Agent installed and running"
    "If someone removes CloudWatch Agent — State Manager reinstalls it"

State Manager = SSM Associations:
├── Association:     a scheduled execution of an SSM document on targets
├── Schedule:        cron or rate expression
├── Compliance:      reports COMPLIANT/NON_COMPLIANT per instance
└── Auto-remediation: re-applies if instance drifts from desired state

Common State Manager use cases:
├── Ensure CloudWatch Agent installed + configured on all instances
├── Ensure SSM Agent is always updated to latest version
├── Apply security configurations (disable root login, configure sudoers)
├── Ensure required software packages always installed
└── Keep configuration files in sync with Parameter Store values

Creating a State Manager association:
# Always keep CloudWatch Agent running on all prod instances
aws ssm create-association \
```

```
    --name "AWS-ConfigureAWSPackage" \
    --association-name "ensure-cw-agent-installed" \
    --targets '[{"Key":"tag:Environment","Values":["prod"]}]' \
    --parameters '{
      "action": ["Install"],
      "name":   ["AmazonCloudWatchAgent"]
    }' \
    --schedule-expression "rate(1 day)" \
    --compliance-severity "HIGH" \
    --apply-only-at-cron-interval false
    # apply-only-at-cron-interval=false → apply immediately on new instances

  # Configure CloudWatch Agent using Parameter Store config
  aws ssm create-association \
    --name "AmazonCloudWatch-ManageAgent" \
    --association-name "configure-cw-agent" \
    --targets '[{"Key":"tag:Environment","Values":["prod"]}]' \
    --parameters '{
      "action":           ["configure"],
      "mode":             ["ec2"],
      "optionalConfigurationSource": ["ssm"],
      "optionalConfigurationLocation": ["/cloudwatch-agent/config/prod"]
    }' \
    --schedule-expression "rate(12 hours)"
    # Every 12 hours, re-applies CW Agent config from Parameter Store
    # If someone manually edited config on instance → restored after 12h

  Compliance checking:
  aws ssm list-compliance-items \
    --filters '[{"Key":"ComplianceType","Values":["Association"]},
                {"Key":"Status","Values":["NON_COMPLIANT"]}]'
  # Returns: which instances have non-compliant associations
```

---

## 💬 Short Crisp Interview Answer

*"Patch Manager, Inventory, and State Manager are SSM's fleet operations capabilities. Patch Manager uses patch baselines to define which patches are approved, groups instances with Patch Group tags, and schedules patching through maintenance windows with rate control and error thresholds — rolling patches to 25% of instances at a time prevents a bad patch from taking down everything. Inventory collects software, configuration, and hardware data from every instance on a schedule, ships to S3 for Athena analysis — answer questions like 'which instances run nginx below version 1.24?' State Manager is ongoing configuration enforcement: associations define desired state (CloudWatch Agent must be installed, SSM Agent must be current), run on a schedule, and re-apply if instances drift — it's the difference between one-time Run Command and continuous configuration management."*

---

## ⚠ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|

| Gotcha | Detail |
|---|---|
| Patch Manager doesn't auto-reboot | Must set RebootOption=RebootIfNeeded explicitly or kernel patches don't activate |
| Kernel patches need a reboot | Installing a kernel update requires reboot to take effect — plan maintenance windows accordingly |
| Inventory sync to S3 has delay | Resource Data Sync to S3 can take up to 30 minutes — not real-time |
| State Manager re-applies on schedule | If human manually makes a change, it gets overwritten at next association execution |
| Maintenance window cutoff | Cutoff (e.g., 1 hour before window ends) stops new tasks — running tasks complete |
| Patch baseline default per OS | Each OS has an AWS-provided default baseline — create custom baselines for production |

---

# 10.3 Terraform on AWS — State Management with S3 + DynamoDB

---

## 🟢 What It Is in Simple Terms

Terraform is the industry-standard Infrastructure as Code tool — you describe your desired AWS infrastructure in HCL (HashiCorp Configuration Language) and Terraform creates, updates, and destroys resources to match. On AWS, the critical operational concern is state management: Terraform tracks what it has deployed in a state file, and in team environments that state must be stored remotely (S3) with locking (DynamoDB) to prevent conflicts.

---

## 🔍 Why Remote State Exists / What Problem It Solves

```
Local state file problem (single developer):
terraform.tfstate: JSON file on your laptop
→ Works for one person
→ Colleague runs terraform apply → reads DIFFERENT local state
→ Thinks nothing exists → tries to create everything → CONFLICT + ERRORS
→ Two people cannot safely work on same infrastructure

Remote state solution (S3 + DynamoDB):
├── S3 bucket: stores terraform.tfstate (single source of truth)
├── DynamoDB table: distributed lock (only one apply at a time)
└── Team: everyone reads/writes same state → no conflicts

State locking problem without DynamoDB:
```

```
Person A: terraform apply (reads state, calculates plan)
Person B: terraform apply (reads SAME state simultaneously)
Person A: writes changes (AWS resources created)
Person B: writes CONFLICTING changes (based on stale state)
→ State file corrupted, resources orphaned, infrastructure broken
```

---

## ⚙️ Remote State Configuration

```
# backend.tf — configure remote state
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  backend "s3" {
    bucket         = "my-company-terraform-state"
    key            = "prod/us-east-1/vpc/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true                                    # SSE-S3
encryption
    kms_key_id     = "arn:aws:kms:us-east-1:123:key/..."      # optional
KMS CMK
    dynamodb_table = "terraform-state-lock"                   # lock table
name
    role_arn       = "arn:aws:iam::123:role/terraform-state"  # cross-
account state
  }
}
```

```
# Create the S3 bucket for state (do this once manually or with a
bootstrap script)
aws s3api create-bucket \
  --bucket my-company-terraform-state \
  --region us-east-1

aws s3api put-bucket-versioning \
  --bucket my-company-terraform-state \
  --versioning-configuration Status=Enabled
# Versioning = every state change is preserved
# Can roll back to previous state version if Terraform corrupts state

aws s3api put-bucket-encryption \
  --bucket my-company-terraform-state \
```

```
  --server-side-encryption-configuration '{
    "Rules": [{
      "ApplyServerSideEncryptionByDefault": {
        "SSEAlgorithm": "aws:kms"
      }
    }]
  }'

aws s3api put-public-access-block \
  --bucket my-company-terraform-state \
  --public-access-block-configuration \
    "BlockPublicAcls=true,IgnorePublicAcls=true,\
     BlockPublicPolicy=true,RestrictPublicBuckets=true"

# Create DynamoDB table for state locking
aws dynamodb create-table \
  --table-name terraform-state-lock \
  --billing-mode PAY_PER_REQUEST \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH

# DynamoDB lock record:
# LockID = "my-company-terraform-state/prod/us-east-
1/vpc/terraform.tfstate"
# Info = {who holds lock, when acquired, operation}
# Terraform automatically creates/deletes this record during plan/apply
```

---

## 🧩 State File Structure and Operations

```
terraform.tfstate structure (simplified):
{
  "version": 4,
  "terraform_version": "1.5.7",
  "serial": 42,          ← increments with every change
  "lineage": "abc-123",  ← unique ID for this state (prevents mixing
states)
  "outputs": {},
  "resources": [
    {
      "mode": "managed",
      "type": "aws_vpc",
      "name": "main",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [{
        "schema_version": 1,
        "attributes": {
          "id":         "vpc-0abc123",
          "cidr_block": "10.0.0.0/16",
          "tags": {"Name": "prod-vpc", "Environment": "prod"}
        }
```

```
      }]
    }
  ]
}

State commands:
# List all resources Terraform is tracking
terraform state list

# Show details of a specific resource
terraform state show aws_vpc.main

# Import existing resource into state (if created outside Terraform)
terraform import aws_vpc.main vpc-0abc123

# Remove resource from state (without destroying it)
terraform state rm aws_vpc.main

# Move resource in state (for refactoring)
terraform state mv aws_vpc.main aws_vpc.production

# Pull remote state locally for inspection
terraform state pull > current_state.json
```

---

## 🧩 Workspaces and Multi-Environment Patterns

```
Pattern 1: Terraform Workspaces (state isolation per environment)
# Workspaces = separate state files for same config
terraform workspace new prod
terraform workspace new staging
terraform workspace select prod
terraform apply   # writes to prod state file

S3 state paths with workspaces:
├── env:/prod/terraform.tfstate
└── env:/staging/terraform.tfstate

Workspaces limitations:
├── Same code for all environments (hard to differ resource sizes)
└── Does not provide IAM isolation between environments

Pattern 2: Separate directories (most common in production)
infrastructure/
├── modules/              ← reusable modules
│   ├── vpc/
│   ├── eks-cluster/
│   └── rds/
├── environments/
│   ├── prod/
│   │   ├── backend.tf    ← points to prod state key
```

```
│   │   ├── main.tf       ← calls modules with prod vars
│   │   └── variables.tf
│   └── staging/
│       ├── backend.tf    ← points to staging state key
│       └── main.tf
└── global/
    ├── iam/
    └── route53/

S3 state key strategy:
├── prod/us-east-1/vpc/terraform.tfstate
├── prod/us-east-1/eks/terraform.tfstate
├── prod/us-east-1/rds/terraform.tfstate
└── staging/us-east-1/vpc/terraform.tfstate
# Separate state per component = blast radius control
# Breaking staging vpc doesn't touch prod

Pattern 3: Remote state data source (read another component's outputs)
# eks/main.tf reads VPC outputs from vpc state
data "terraform_remote_state" "vpc" {
  backend = "s3"
  config = {
    bucket = "my-company-terraform-state"
    key    = "prod/us-east-1/vpc/terraform.tfstate"
    region = "us-east-1"
  }
}

resource "aws_eks_cluster" "main" {
  vpc_config {
    subnet_ids =
data.terraform_remote_state.vpc.outputs.private_subnet_ids
  }
}
# EKS reads exact subnet IDs from VPC state → no hardcoded IDs
```

---

## 🧩 Terraform AWS Provider Best Practices

```
# provider.tf
provider "aws" {
  region = var.aws_region

  # Assume role for each environment (avoid using long-lived credentials)
  assume_role {
    role_arn     = "arn:aws:iam::${var.account_id}:role/terraform-
deployer"
    session_name = "terraform-${var.environment}"
    duration_seconds = 3600
  }
```

```
  default_tags {
    tags = {
      ManagedBy   = "Terraform"
      Environment = var.environment
      Repository  = "github.com/company/infrastructure"
    }
  }
}

# Multiple providers for multi-region/multi-account
provider "aws" {
  alias  = "us_west_2"
  region = "us-west-2"
  assume_role {
    role_arn = "arn:aws:iam::${var.account_id}:role/terraform-deployer"
  }
}

# Use alternate provider for specific resources
resource "aws_s3_bucket" "dr_bucket" {
  provider = aws.us_west_2
  bucket   = "my-dr-bucket"
}
```

```
IAM permissions for Terraform deployer role:
├── Development: broad permissions (PowerUserAccess or
AdministratorAccess)
├── Production: least privilege (only services Terraform manages)
└── State bucket: read/write state key, DynamoDB lock table read/write

Trust policy for CI/CD (GitHub Actions OIDC — no long-lived keys):
{
  "Principal": {
    "Federated": "arn:aws:iam::123:oidc-
provider/token.actions.githubusercontent.com"
  },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
    },
    "StringLike": {
      "token.actions.githubusercontent.com:sub":
        "repo:mycompany/infrastructure:ref:refs/heads/main"
    }
  }
}
# GitHub Actions assumes AWS role via OIDC — zero AWS access keys stored
in GitHub
```

## 💬 Short Crisp Interview Answer

*"Terraform manages AWS infrastructure as code with HCL. In team environments, state must be stored remotely in S3 with DynamoDB locking. S3 holds the state file with versioning enabled (every change preserved, rollback possible), encrypted at rest. DynamoDB provides distributed locking — one DynamoDB item per state file, created at the start of plan/apply and deleted after. Without DynamoDB locking, two simultaneous applies corrupt state. The state key strategy is important: separate state per component (vpc/, eks/, rds/) so a failure in one component doesn't affect others. Use Terraform remote state data sources to reference outputs across component boundaries. For CI/CD, use GitHub Actions OIDC to assume an IAM role — no long-lived AWS access keys stored in CI secrets."*

---

## ⚠️ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| Never edit state file manually | Manual edits corrupt state lineage and serial number — use terraform state commands |
| State lock leftover | If apply crashes mid-execution, lock persists — manually delete DynamoDB item with terraform force-unlock |
| terraform destroy in wrong workspace | Destroying wrong workspace = deleting wrong environment. Always verify workspace before destroy |
| Sensitive values in state | Passwords and secrets in Terraform resources are stored in plaintext in state file — encrypt S3 |
| Resource drift | Someone manually changes AWS resource — Terraform state and reality differ. Run terraform plan to detect |
| Import doesn't generate config | terraform import adds to state but doesn't write .tf files — must write config manually |

---

# 10.4 AWS CDK — Constructs, Stacks, Bootstrapping

---

## 🟢 What It Is in Simple Terms

AWS CDK (Cloud Development Kit) lets you define AWS infrastructure using real programming languages — TypeScript, Python, Java, Go, C# — instead of JSON/YAML templates. CDK compiles your code into CloudFormation templates behind the scenes. The advantage: use loops, conditionals, inheritance, and existing package managers to write infrastructure with the full power of a programming language.

---

## 🔍 CDK vs CloudFormation vs Terraform

```
┌──────────────────┬──────────────────┬──────────────────┬──────────────────┐
│ Feature          │ CDK              │ CloudFormation   │ Terraform        │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ Language         │ TypeScript,      │ JSON / YAML      │ HCL              │
│                  │ Python, Java,    │ (declarative)    │ (declarative)    │
│                  │ Go, C#           │                  │                  │
│ Expressiveness   │ Full language    │ Limited          │ Medium           │
│ Abstraction      │ L1/L2/L3         │ Raw resources    │ Modules          │
│ AWS-native       │ ✅ Yes           │ ✅ Yes           │ ❌ (plugin)      │
│ Multi-cloud      │ ❌ AWS only      │ ❌ AWS only      │ ✅ Yes           │
│ State management │ CloudFormation   │ CloudFormation   │ Custom (S3)      │
│ Compile step     │ cdk synth        │ None             │ None             │
│ Drift detection  │ ✅ CF detects    │ ✅ CF detects    │ terraform plan   │
└──────────────────┴──────────────────┴──────────────────┴──────────────────┘

CDK strengths:
├── Write infrastructure in familiar language (TypeScript is most popular)
├── Reuse existing package managers (npm, pip) for CDK constructs
├── Type safety catches errors at compile time, not deploy time
├── Higher-level abstractions (L2/L3 constructs) reduce boilerplate
└── Generate dynamic resource counts with for loops

CDK compiles to CloudFormation — CloudFormation handles:
├── Deployment ordering (dependency graph)
├── Rollback on failure
├── Drift detection
└── Change sets (preview before apply)
```

---

## 🧩 Constructs — The Building Blocks

```
CDK has three levels of constructs:

L1 Constructs (Cfn* classes — CloudFormation resources directly):
├── 1:1 mapping to CloudFormation resource types
├── All CloudFormation properties exposed — very verbose
└── Use when: no L2 exists for a service, need full control

// L1: CfnBucket — ALL properties must be set manually
import { aws_s3 as s3 } from 'aws-cdk-lib';
new s3.CfnBucket(this, 'MyBucket', {
  bucketName: 'my-bucket',
  versioningConfiguration: { status: 'Enabled' },
  bucketEncryption: {
    serverSideEncryptionConfiguration: [{
      serverSideEncryptionByDefault: { sseAlgorithm: 'AES256' }
    }]
  },
  publicAccessBlockConfiguration: {
    blockPublicAcls: true,
```

```
    blockPublicPolicy: true,
    ignorePublicAcls: true,
    restrictPublicBuckets: true
  }
});

L2 Constructs (the CDK sweet spot — opinionated defaults):
├── Higher-level abstractions with secure defaults built in
├── Encryption, logging, versioning often enabled by default
├── Methods like addLifecycleRule(), grantRead(), addEventNotification()
└── Use for: most standard resource configurations

// L2: Bucket — secure defaults, expressive API
const bucket = new s3.Bucket(this, 'MyBucket', {
  versioned:          true,
  encryption:         s3.BucketEncryption.S3_MANAGED,
  blockPublicAccess:  s3.BlockPublicAccess.BLOCK_ALL,
  removalPolicy:      RemovalPolicy.RETAIN,       // don't delete on stack
destroy
  autoDeleteObjects:  false,
});

// Grant read permission to a Lambda (generates correct IAM policy
automatically)
bucket.grantRead(myLambda);

// Add lifecycle rule
bucket.addLifecycleRule({
  transitions: [{
    storageClass: s3.StorageClass.INTELLIGENT_TIERING,
    transitionAfter: Duration.days(30)
  }]
});

L3 Constructs (Patterns — complete solutions):
├── Multi-resource solutions for common architectures
├── Example: ApplicationLoadBalancedFargateService
│   Creates: ECS cluster, task definition, Fargate service, ALB,
│            security groups, IAM roles, target groups — everything
└── Use for: standard architectures where conventions are acceptable

// L3: Complete ECS Fargate + ALB setup in ~10 lines
import { ApplicationLoadBalancedFargateService } from 'aws-cdk-lib/aws-
ecs-patterns';

new ApplicationLoadBalancedFargateService(this, 'MyService', {
  cluster: myCluster,
  cpu: 256,
  memoryLimitMiB: 512,
  desiredCount: 3,
  taskImageOptions: {
    image: ecs.ContainerImage.fromRegistry('amazon/amazon-ecs-sample')
  },
  publicLoadBalancer: true
```

```
});
// CDK generates: ALB, listener, target group, ECS service, task def,
//                IAM roles, security groups, CloudWatch alarms
```

---

## 🧩 Stacks and Apps

```
CDK App → contains one or more Stacks → each Stack = one CloudFormation
stack

CDK App structure:
my-cdk-app/
├── bin/
│   └── app.ts           ← entry point, creates stacks
├── lib/
│   ├── vpc-stack.ts     ← VPC infrastructure
│   ├── eks-stack.ts     ← EKS cluster (imports VPC outputs)
│   └── rds-stack.ts     ← RDS (imports VPC + EKS outputs)
├── test/
│   └── app.test.ts      ← unit tests for infrastructure
├── cdk.json             ← CDK app configuration
└── package.json
```

```typescript
// bin/app.ts — creates stacks with explicit account/region
import * as cdk from 'aws-cdk-lib';
import { VpcStack } from '../lib/vpc-stack';
import { EksStack } from '../lib/eks-stack';

const app = new cdk.App();

// Environment-specific stacks
const env = {
  account: process.env.CDK_DEFAULT_ACCOUNT,
  region:  process.env.CDK_DEFAULT_REGION
};

const vpcStack = new VpcStack(app, 'ProdVpcStack', {
  env,
  stackName: 'prod-vpc',
  description: 'Production VPC infrastructure'
});

const eksStack = new EksStack(app, 'ProdEksStack', {
  env,
  vpc: vpcStack.vpc  // pass VPC reference directly — CDK handles
dependency ordering
});
```

```
// lib/vpc-stack.ts — VpcStack construct
import * as cdk from 'aws-cdk-lib';
import * as ec2 from 'aws-cdk-lib/aws-ec2';

export class VpcStack extends cdk.Stack {
  public readonly vpc: ec2.Vpc;  // export for other stacks

  constructor(scope: cdk.App, id: string, props: cdk.StackProps) {
    super(scope, id, props);

    this.vpc = new ec2.Vpc(this, 'ProdVpc', {
      maxAzs: 3,
      natGateways: 1,
      subnetConfiguration: [
        { subnetType: ec2.SubnetType.PUBLIC,  name: 'Public',  cidrMask:
24 },
        { subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS, name: 'Private',
cidrMask: 24 },
        { subnetType: ec2.SubnetType.PRIVATE_ISOLATED,    name:
'Isolated', cidrMask: 28 }
      ]
    });

    // Outputs (visible in CloudFormation console + usable by other
stacks)
    new cdk.CfnOutput(this, 'VpcId', { value: this.vpc.vpcId });
  }
}
```

```
CDK commands:
├── cdk synth:    compile → generate CloudFormation template
│                 (no AWS calls — safe to run anytime)
├── cdk diff:     compare with deployed stack (like terraform plan)
├── cdk deploy:   synthesize + deploy to AWS (creates/updates CF stack)
├── cdk destroy:  delete stack and all its resources
├── cdk ls:       list all stacks in the app
└── cdk doctor:   check CDK environment + dependencies

CDK context:
├── Stores runtime lookups (AZ names, AMI IDs, SSM parameter values)
├── Cached in cdk.context.json (commit to git for reproducible builds)
└── Clear with: cdk context --clear

Testing CDK constructs (jest):
import { Template } from 'aws-cdk-lib/assertions';
import { VpcStack } from '../lib/vpc-stack';

test('VPC has correct CIDR and 3 AZs', () => {
  const app = new cdk.App();
  const stack = new VpcStack(app, 'TestStack');
  const template = Template.fromStack(stack);
```

```
  template.hasResourceProperties('AWS::EC2::VPC', {
    CidrBlock: '10.0.0.0/16'
  });
  template.resourceCountIs('AWS::EC2::Subnet', 9); // 3 types × 3 AZs
});
```

---

## 🧩 CDK Bootstrapping

```
CDK Bootstrap = preparing an AWS account+region for CDK deployments

Why bootstrapping is needed:
├── CDK assets (Lambda zip files, Docker images) must be stored somewhere
├── CDK needs an S3 bucket to store asset files during deployment
├── CDK needs an ECR repo for Docker image assets
└── CDK needs an IAM role for CloudFormation to execute with

Bootstrap creates (in your account):
├── S3 bucket:    cdk-xxxxxxxxxx-assets-{account}-{region}
│                 Stores: Lambda zip files, CloudFormation templates
├── ECR repo:     cdk-xxxxxxxxxx-container-assets-{account}-{region}
│                 Stores: Docker images for ECS/Lambda container assets
├── IAM roles:
│   ├── cdk-xxxxxxxxxx-deploy-role:      CloudFormation execution role
│   ├── cdk-xxxxxxxxxx-lookup-role:      reads from account for cdk
diff/synth
│   ├── cdk-xxxxxxxxxx-image-publishing-role: pushes to ECR
│   └── cdk-xxxxxxxxxx-file-publishing-role:  uploads to S3 bootstrap
bucket
└── SSM Parameter: /cdk-bootstrap/{qualifier}/version

Running bootstrap:
# Bootstrap a single account+region
cdk bootstrap aws://123456789012/us-east-1

# Bootstrap with custom qualifier (for multiple CDK environments in same
account)
cdk bootstrap --qualifier myapp aws://123456789012/us-east-1

# Cross-account deployment (trust CI/CD account to deploy to prod)
cdk bootstrap \
  --trust 111111111111 \
  --cloudformation-execution-policies
arn:aws:iam::aws:policy/AdministratorAccess \
  aws://222222222222/us-east-1
# Account 111111111111 (CI/CD) can now deploy into account 222222222222
(prod)

Bootstrap stack name: CDKToolkit (in CloudFormation)
Bootstrap version tracked in SSM parameter — CDK checks version
```

```
compatibility

⚠ Bootstrap once per account+region combination.
   If you have 3 regions × 5 accounts = 15 bootstrap runs.
   Bootstrap must be re-run if CDK major version upgrade requires new
roles.
```

---

## 💬 Short Crisp Interview Answer

*"CDK lets you write AWS infrastructure in TypeScript, Python, or other languages and compiles to CloudFormation. Constructs are the building blocks: L1 maps 1:1 to CloudFormation resources (verbose), L2 adds secure defaults and expressive APIs (grantRead(), addLifecycleRule()), and L3 patterns create complete multi-resource architectures like ApplicationLoadBalancedFargateService. Stacks are the deployment units — one Stack becomes one CloudFormation stack. Cross-stack references work by passing construct references directly. Bootstrapping prepares an account+region by creating the S3 bucket for assets, ECR repo for container images, and IAM roles CDK uses — run cdk bootstrap once per account+region. The big advantage over raw CloudFormation: loops, conditionals, type safety, unit testing with jest, and npm packages for reusable infrastructure components."*

---

## ⚠ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| cdk synth makes AWS API calls | Context lookups (AZ names, AMI IDs) make real AWS API calls during synth — needs credentials |
| Bootstrap per region | Bootstrap must run in EVERY region you deploy to — easy to forget |
| Cross-stack reference limitation | Cannot delete a stack if another stack references its outputs — remove reference first |
| Stack name changes = new stack | Changing CDK stack ID creates a new stack and deletes old one — potential data loss |
| RemovalPolicy.RETAIN | Default for databases/buckets — stack deletion doesn't delete the resource |
| CDK version mismatch | CDK library version in code must match CDK CLI version — use `npm update aws-cdk` |

---

# 10.5 Service Control Policies — Guardrails at Org Level

---

## 🟢 What It Is in Simple Terms

SCPs are AWS Organizations-level policies that act as guardrails for all accounts in an organizational unit or the entire organization. They define the maximum permissions any principal in an account can have — even

the root user. SCPs don't grant permissions, they only restrict them.

*(SCPs were introduced in Category 8 in the context of IAM evaluation. Here we go deeper into operational patterns for platform teams.)*

---

## 🔍 SCPs vs IAM Policies — The Key Distinction

```
IAM Policy:   grants permissions to a specific user/role
SCP:          restricts what any user/role in an account can EVER do

SCP + IAM are cumulative restrictions (AND logic):
An action is allowed ONLY IF:
├── SCP allows it (or no SCP restricts it) AND
└── IAM policy grants it

An action is DENIED if:
├── SCP explicitly denies it (regardless of IAM) OR
└── SCP doesn't allow it (no explicit allow in SCP = deny) OR
└── IAM doesn't grant it

Default SCP: FullAWSAccess (allows everything — no restriction)
All accounts start with this — you layer restrictions on top.
```

---

## 🧩 OU Structure and SCP Design

```
Organization structure with SCP layers:

Root
├── SCP: Baseline security (applied to ALL accounts)
│   - Deny disabling GuardDuty, CloudTrail, Config
│   - Deny root user API access except with MFA
│
├── OU: Infrastructure
│   ├── SCP: Allow all regions + specialized services
│   └── Account: network (Transit Gateway, Route53 hub)
│
├── OU: Production
│   ├── SCP: Deny non-approved regions
│   ├── SCP: Require encryption on all resources
│   ├── SCP: Deny large instance types (cost control)
│   └── Account: prod-us-east-1
│
├── OU: Staging
│   ├── SCP: Deny production-scale resources (cost control)
│   └── Account: staging-us-east-1
│
└── OU: Development
    ├── SCP: Deny expensive services (Redshift, SageMaker training)
```

```
    ├── SCP: Deny operations in non-dev regions
    └── Accounts: dev-alice, dev-bob, sandbox-team-x

Key principle: SCPs are cumulative from root → OU → account.
Account gets intersection of ALL ancestor SCPs.
```

---

## 🧩 Production SCP Examples

```
// 1. BASELINE: Protect security services from being disabled
// Apply at Root OU — protects ALL accounts
{
  "Sid": "ProtectSecurityServices",
  "Effect": "Deny",
  "Action": [
    "guardduty:DeleteDetector",
    "guardduty:DisassociateFromMasterAccount",
    "cloudtrail:DeleteTrail",
    "cloudtrail:StopLogging",
    "cloudtrail:UpdateTrail",
    "config:DeleteConfigurationRecorder",
    "config:DeleteDeliveryChannel",
    "securityhub:DisableSecurityHub",
    "securityhub:DeleteHub",
    "access-analyzer:DeleteAnalyzer"
  ],
  "Resource": "*",
  "Condition": {
    "ArnNotLike": {
      "aws:PrincipalArn": "arn:aws:iam::*:role/SecurityBreakGlassRole"
    }
  }
}
// Exception: SecurityBreakGlassRole can override — for legitimate
security ops

// 2. PROD: Enforce encryption on EBS volumes and RDS instances
{
  "Sid": "RequireEBSEncryption",
  "Effect": "Deny",
  "Action": ["ec2:RunInstances"],
  "Resource": "arn:aws:ec2:*:*:volume/*",
  "Condition": {
    "Bool": {"ec2:Encrypted": "false"}
  }
}

{
  "Sid": "RequireRDSEncryption",
  "Effect": "Deny",
  "Action": ["rds:CreateDBInstance", "rds:CreateDBCluster"],
```

```
    "Resource": "*",
    "Condition": {
      "Bool": {"rds:StorageEncrypted": "false"}
    }
  }

  // 3. PROD: Restrict to approved regions only
  {
    "Sid": "DenyNonApprovedRegions",
    "Effect": "Deny",
    "NotAction": [
      "iam:*",
      "organizations:*",
      "route53:*",
      "cloudfront:*",
      "waf:*",
      "sts:*",
      "support:*"
    ],
    "Resource": "*",
    "Condition": {
      "StringNotEquals": {
        "aws:RequestedRegion": ["us-east-1", "us-east-2", "eu-west-1"]
      }
    }
  }
  // NotAction: global services (IAM, Route53, CloudFront) are always
  allowed
  // Everything else: denied outside approved regions

  // 4. DEV: Deny expensive instance types and services
  {
    "Sid": "DenyExpensiveEC2",
    "Effect": "Deny",
    "Action": "ec2:RunInstances",
    "Resource": "arn:aws:ec2:*:*:instance/*",
    "Condition": {
      "StringLike": {
        "ec2:InstanceType": [
          "*.24xlarge", "*.18xlarge", "*.16xlarge",
          "p3.*", "p4.*", "inf1.*", "trn1.*"
        ]
      }
    }
  }

  {
    "Sid": "DenyExpensiveServices",
    "Effect": "Deny",
    "Action": [
      "redshift:CreateCluster",
      "sagemaker:CreateTrainingJob",
      "sagemaker:CreateEndpoint"
    ],
```

```
  "Resource": "*"
}

// 5. PROD: Prevent accidental S3 bucket deletion
{
  "Sid": "ProtectProdS3Buckets",
  "Effect": "Deny",
  "Action": [
    "s3:DeleteBucket",
    "s3:DeleteBucketPolicy"
  ],
  "Resource": "arn:aws:s3:::prod-*",
  "Condition": {
    "ArnNotLike": {
      "aws:PrincipalArn": "arn:aws:iam::*:role/InfrastructureAdminRole"
    }
  }
}
```

---

## 🧩 SCP Management at Scale

```
# Create SCP
aws organizations create-policy \
  --name "prod-require-encryption" \
  --description "Require encryption on all data services" \
  --type SERVICE_CONTROL_POLICY \
  --content file://prod-encryption-scp.json

# Attach SCP to OU
aws organizations attach-policy \
  --policy-id p-abc123 \
  --target-id ou-xyz-prod123

# Attach SCP to specific account
aws organizations attach-policy \
  --policy-id p-abc123 \
  --target-id 123456789012

# List SCPs on an OU
aws organizations list-policies-for-target \
  --target-id ou-xyz-prod123 \
  --filter SERVICE_CONTROL_POLICY

# Simulate SCP impact (use IAM Policy Simulator)
# Or use: aws iam simulate-principal-policy with SCP context
```

```
SCP simulation and testing:
├── IAM Policy Simulator: test SCP + IAM combination
```

```
  ├── AWS CloudShell in affected account: test denied actions
  └── cdk/terraform policy testing: validate SCPs before attaching to prod

  SCP best practices:
  ├── Start permissive → add restrictions iteratively
  ├── Test in sandbox OU before attaching to prod OU
  ├── Use a break-glass exception (specific IAM role can bypass)
  ├── Document every SCP with purpose + date + owner
  └── Review SCPs during org changes (new accounts, new services)

  ⚠ SCPs apply to ALL principals including root.
     If SCP denies ec2:RunInstances, even root cannot launch instances.
     Break-glass role (excluded from SCP deny) is critical for recovery.
```

---

## 💬 Short Crisp Interview Answer

*"SCPs are org-level guardrails that restrict what any principal — including root — in an account can do, regardless of their IAM permissions. An SCP deny cannot be overridden by any IAM allow. They're layered from root through OUs to accounts, so an account inherits all ancestor SCPs cumulatively. In production I apply SCPs in three layers: a root SCP protecting security services from being disabled (GuardDuty, CloudTrail, Config), a prod OU SCP restricting regions and requiring encryption, and a dev OU SCP denying expensive instance types and services. A break-glass exception pattern excludes a specific SecurityAdminRole from the deny conditions — critical for legitimate security operations without bypassing all SCPs. Always test SCPs in a sandbox OU before attaching to production."*

---

## ⚠ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| NotAction for global services | Region restriction SCP must use NotAction for IAM, Route53, CloudFront — these are global |
| SCP on management account | SCPs don't apply to the management (root) account — use it sparingly |
| Break-glass is critical | A bad SCP with no break-glass exception can permanently lock out an account |
| Service-linked roles exempt | Service-linked roles are not restricted by SCPs — AWS service operations bypass SCPs |
| SCP doesn't grant | Having FullAWSAccess SCP doesn't grant anything — IAM must still grant the permission |

---

# 10.6 AWS Control Tower — Landing Zones, Guardrails

---

## 🟢 What It Is in Simple Terms

AWS Control Tower is the managed service for setting up and governing a multi-account AWS environment (called a landing zone). It automates what would take weeks of manual setup: creating a well-architected account structure, applying security guardrails, setting up centralized logging, and enabling federated identity — all following AWS best practices out of the box.

---

## 🔍 Why Control Tower Exists / What Problem It Solves

```
Setting up a new AWS organization manually:
├── Create Organizations structure (Root, OUs, accounts)
├── Enable CloudTrail in all accounts (manually)
├── Enable Config in all accounts (manually)
├── Set up centralized logging account + S3 bucket
├── Create Identity account for SSO
├── Write all the SCPs (days of work)
├── Set up AWS SSO / IAM Identity Center
├── Enroll each new account into all the above
└── Repeat for every new account created → weeks of work

Control Tower automates all of this:
├── Creates the account structure automatically
├── Deploys guardrails (SCPs + Config rules) across all accounts
├── Sets up centralized logging to dedicated log archive account
├── Integrates AWS IAM Identity Center (SSO) out of the box
└── Account Factory: provision new accounts in minutes following standard
```

---

## 🧩 Control Tower Landing Zone

```
Landing Zone = the governed multi-account AWS environment
               that Control Tower sets up and manages

Accounts created by Control Tower:
├── Management account:  root of organization, Control Tower itself
├── Log Archive account: centralized CloudTrail + Config logs from all
accounts
│   └── S3 bucket receives all logs — read-only for most users
├── Audit account:       security team gets read-only access to ALL
accounts
│   └── SNS topics receive compliance alerts from all accounts
└── Your workload accounts: provisioned via Account Factory

Control Tower OU structure (opinionated default):
Root
├── Management Account (Control Tower master)
├── Security OU
│   ├── Log Archive Account
```

```
│   └── Audit Account
├── Sandbox OU (for experimentation)
├── Workloads OU
│   ├── Production OU
│   │   └── Workload accounts...
│   └── Staging OU
│       └── Workload accounts...
└── Suspended OU (for decommissioned accounts)

What gets deployed automatically in every enrolled account:
├── CloudTrail: organization trail logs to Log Archive account
├── AWS Config: enabled, delivers to Log Archive account
├── Config rules: Control Tower detective guardrails
├── IAM Identity Center: SSO permission sets provisioned
└── SNS: compliance notifications to Audit account
```

---

## 🧩 Guardrails

```
Guardrails = policies that govern your landing zone
             Two types: Preventive and Detective

Preventive Guardrails = SCPs that prevent non-compliant actions
├── Status: Enforced (SCP attached) or Not Enabled
└── Examples:
    - Disallow changes to CloudTrail logging
    - Disallow deletion of log archive S3 bucket
    - Require encryption for EBS volumes
    - Disallow creation of access keys for root user

Detective Guardrails = AWS Config rules that detect non-compliance
├── Status: Enabled (Config rule running) or Not Enabled
├── Reports: COMPLIANT / NON_COMPLIANT per account
└── Examples:
    - Detect whether MFA is enabled for root
    - Detect whether S3 buckets have public access
    - Detect whether EC2 security groups allow unrestricted SSH
    - Detect whether EBS volumes are encrypted

Guardrail categories:
├── Mandatory:  always enforced, cannot disable (baseline security)
├── Strongly Recommended: enabled by default, can disable
└── Elective:   opt-in guardrails for additional controls

Mandatory guardrail examples (cannot disable):
├── Disallow changes to CloudTrail in enrolled accounts
├── Disallow deletion of log archive account
├── Disallow changes to IAM Identity Center setup
└── Disallow changes to the landing zone account structure

Checking guardrail compliance:
```

```
aws controltower list-enabled-controls \
  --target-identifier arn:aws:organizations::123:ou/o-xxx/ou-yyy

# Control Tower dashboard shows:
# Per account: drift status, guardrail compliance status
# Per guardrail: how many accounts compliant vs non-compliant
```

---

## 🧩 Account Factory

```
Account Factory = self-service account provisioning through Control Tower

Account Factory capabilities:
├── Provision new AWS accounts following standard template
├── New account gets: correct OU placement, all guardrails applied,
│                     SSO permission sets, CloudTrail, Config —
automatically
├── Account vending machine for teams (via Service Catalog)
└── Account Factory for Terraform (AFT): code-based account provisioning

Without Account Factory: provisioning a new account takes days.
With Account Factory:    provisioning a new account takes ~30 minutes.

Account Factory for Terraform (AFT):
├── Define account template in Terraform
├── Git PR triggers AFT pipeline
└── AFT provisions account + applies customizations + deploys baseline

# Account Factory workflow:
1. Team submits Account Factory product through Service Catalog
2. Provides: account name, email, OU, SSO permission sets
3. Control Tower creates account in correct OU
4. All guardrails auto-applied to new account
5. CloudTrail + Config auto-enabled and shipping to Log Archive
6. SSO permission sets auto-provisioned
7. Team gets account access in ~30 minutes

Customizations with CfCT (Customizations for Control Tower):
├── Deploy additional resources to every new account automatically
│   (VPC, security groups, IAM roles, etc.)
├── Git-based: store customizations in CodeCommit/GitHub
└── Pipeline: triggered when new account enrolled → applies customizations
```

---

## 🧩 Control Tower Drift

```
Landing Zone Drift = something in the Control Tower setup was changed
                     outside of Control Tower's management
                     (e.g., someone manually deleted an SCP, modified a
```

```
Config rule)

Drift detection:
├── Control Tower detects drift automatically
├── Dashboard shows drift status per account
└── Drift types:
    - DRIFTED: something changed from expected state
    - IN_SYNC: everything matches expected state
    - UPDATE_AVAILABLE: Control Tower has a new version to apply

Resolving drift:
aws controltower reset-landing-zone
# Re-applies all SCPs, Config rules, CloudTrail settings
# Fixes any drift — restores landing zone to expected state
```

---

## 💬 Short Crisp Interview Answer

*"Control Tower automates setting up a well-governed multi-account AWS environment — a landing zone. It creates the org structure with dedicated Log Archive and Audit accounts, enables CloudTrail and Config in all accounts shipping to the Log Archive account centrally, deploys guardrails (preventive SCPs and detective Config rules), and integrates IAM Identity Center for SSO. Guardrails have three tiers: mandatory (always enforced, like protecting CloudTrail), strongly recommended (on by default), and elective (opt-in). Account Factory provisions new accounts in 30 minutes following the standard template — instead of days of manual setup — with all guardrails auto-applied. If someone manually changes something in Control Tower's setup, the dashboard shows drift and you run reset-landing-zone to restore the expected state."*

---

## ⚠ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| Control Tower takes ~1 hour to set up | Initial landing zone deployment takes 60-90 minutes — not instant |
| Cannot enroll existing accounts easily | Enrolling pre-existing accounts with custom configs can cause drift issues |
| Mandatory guardrails cannot be disabled | If a mandatory guardrail conflicts with a legitimate need, workaround required |
| Account Factory email must be unique | Each AWS account requires a unique email address — plan an email naming convention |
| Landing zone region = Control Tower region | Control Tower home region must be decided upfront — changing regions is difficult |
| Drift repair can be disruptive | reset-landing-zone re-applies SCPs which may temporarily block operations |

---

# 10.7 Resource Access Manager (RAM)

---

## 🟢 What It Is in Simple Terms

AWS RAM (Resource Access Manager) lets you share AWS resources between accounts within the same organization or with specific external accounts — without duplicating resources in every account. Instead of creating a Transit Gateway in 10 accounts, create it once and share it with all 10.

---

## 🔍 Why RAM Exists / What Problem It Solves

```
Without RAM:
Multi-account organization, each account needs:
├── VPC: need Transit Gateway for connectivity → create one in EACH
account? NO
├── Route53 Resolver Rules: DNS forwarding rules → duplicate in each
account?
├── License Manager configurations → re-create in each account?
└── Prefix Lists: IP allow/deny lists → maintain copies in 50+ accounts?

With RAM:
Create resource ONCE in network account → share to all member accounts via
RAM.
Each member account sees the shared resource as if it were in their own
account.
One Transit Gateway, shared to 200 accounts → all connect to it.
```

---

## 🧩 Resources Shareable via RAM

```
Shareable resource types (key ones for DevOps):
├── VPC Resources:
│   ├── Subnets (VPC sharing — accounts launch resources into shared
subnets)
│   ├── Transit Gateways and Transit Gateway attachments
│   ├── Transit Gateway route tables
│   └── Customer gateways
├── Networking:
│   ├── Route53 Resolver rules (DNS forwarding)
│   ├── Route53 Resolver DNS Firewall rule groups
│   └── Managed prefix lists (IP address lists)
├── Compute:
│   ├── Capacity Reservations (EC2 reserved capacity, shared across
accounts)
│   └── Dedicated Hosts
├── Security:
│   ├── AWS License Manager configurations
```

```
│   └── AWS Network Firewall policies
├── Data:
│   ├── AWS Glue Data Catalogs
│   └── AWS Resource Groups
└── Developer tools:
    ├── CodeBuild report groups
    └── Image Builder components + recipes + pipelines

Most important RAM use case: VPC Subnets sharing
├── Create VPC in central networking account
├── Share specific subnets to workload accounts
├── Workload accounts launch EC2, RDS, Lambda into SHARED subnets
├── One VPC, many accounts, consistent addressing
└── Network team manages VPC centrally, app teams use it
```

---

## 🧩 RAM in Practice — Transit Gateway Sharing

```
Central networking account:    123456789001
Workload accounts:             123456789002, 123456789003, 123456789004

Step 1: Create Transit Gateway in network account
aws ec2 create-transit-gateway \
  --description "Org-wide connectivity hub" \
  --options '{"AutoAcceptSharedAttachments":"enable"}'

Step 2: Share Transit Gateway via RAM
aws ram create-resource-share \
  --name "org-transit-gateway-share" \
  --resource-arns arn:aws:ec2:us-east-1:123456789001:transit-gateway/tgw-
abc123 \
  --principals arn:aws:organizations::123456789001:organization/o-abc123
  # Share with entire organization — all current + future accounts

Step 3: In workload account (123456789002), create attachment
# The shared TGW appears in the workload account
aws ec2 create-transit-gateway-vpc-attachment \
  --transit-gateway-id tgw-abc123 \   ← shared TGW from network account
  --vpc-id vpc-workload-123 \
  --subnet-ids subnet-a subnet-b subnet-c

Step 4: Route traffic through TGW
aws ec2 create-route \
  --route-table-id rtb-workload-123 \
  --destination-cidr-block 10.0.0.0/8 \
  --transit-gateway-id tgw-abc123
# Workload account routes to TGW for all internal 10.x.x.x traffic

Result:
├── 1 Transit Gateway in network account
├── All workload accounts attached to same TGW
```

```
├── Network team controls routing centrally
└── No Transit Gateway cost in each workload account
```

---

## 🧩 RAM — VPC Subnet Sharing

```
VPC Subnet Sharing = share subnets from one account to others
                     Other accounts launch their resources INTO these
subnets

Network account (123456789001):
├── Creates VPC: 10.0.0.0/16
├── Creates subnets: 10.0.1.0/24, 10.0.2.0/24, 10.0.3.0/24
└── Shares specific subnets to specific workload accounts via RAM

Workload account (123456789002):
├── Sees shared subnets in subnet list
├── Launches EC2 instances into shared subnet
├── Launches RDS into shared subnet
└── Resources get IPs from 10.0.x.x range (managed by network account)

What each account controls:
Network account:
├── VPC CIDR, route tables, internet gateway, NAT gateway
├── Security Groups in VPC (can't be shared — each account creates own
SGs)
└── VPC Flow Logs

Workload account:
├── Their own resources (EC2, RDS, Lambda) in shared subnet
├── Their own Security Groups (attached to their resources)
└── Their own IAM roles and policies

Share subnets:
aws ram create-resource-share \
  --name "prod-private-subnets" \
  --resource-arns \
    arn:aws:ec2:us-east-1:123456789001:subnet/subnet-abc123 \
    arn:aws:ec2:us-east-1:123456789001:subnet/subnet-def456 \
  --principals \
    123456789002 \
    123456789003

Benefits of VPC sharing:
├── Consistent IP addressing across all workload accounts
├── Single VPC to manage (fewer VPC peering connections)
├── Transit Gateway not needed for inter-account in same VPC
└── Network team maintains central control
```

---

## 🧩 RAM with Organizations

```
Sharing with AWS Organizations:
├── Share with entire organization (all current + future accounts):
│   --principals arn:aws:organizations::MGMT-ACCT:organization/o-abc123
├── Share with specific OU (e.g., all prod accounts):
│   --principals arn:aws:organizations::MGMT-ACCT:ou/o-abc123/ou-xyz-prod
└── Share with specific account:
    --principals 123456789012

Organization sharing benefits:
├── New accounts automatically receive shared resources
├── No manual sharing invitation required for org members
└── Revoke access by removing from OU or organization

RAM with Organizations enabled (recommended):
aws ram enable-sharing-with-aws-organization
# Enables sharing without requiring manual invitation acceptance
# For org sharing — new members automatically get resources
```

---

## 💬 Short Crisp Interview Answer

*"RAM lets you share AWS resources across accounts without duplicating them. The most important use cases are Transit Gateway sharing — create one TGW in a central networking account and share it to all 200 accounts in the organization via RAM, eliminating separate TGW costs per account — and VPC subnet sharing — share specific subnets so workload accounts launch resources directly into a centrally managed VPC. When sharing with the organization ARN, new accounts automatically receive shared resources without manual invitation. The key governance point: sharing with an OU is more precise than sharing with the whole org — prod OU gets the prod TGW, dev OU gets the dev TGW. Resources remain in the owner account; sharers only get the ability to reference and use them."*

---

## ⚠️ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| Security Groups not shareable | Security Groups cannot be shared via RAM — each account creates their own SGs |
| Resource stays in owner account | Shared resource costs bill to OWNER account, not the accounts using it |
| Organization sharing requires enabling | Must run aws ram enable-sharing-with-aws-organization before org-level sharing works |
| Subnet sharing: route table ownership | Route tables belong to network account — workload accounts cannot modify them |

| Gotcha | Detail |
|---|---|
| Revocation is immediate | Removing a principal from RAM share immediately removes their access — plan carefully |
| RAM share across organization boundary | Sharing outside organization requires manual invitation + acceptance by recipient account |

---

# 10.8 Tag Policies & Cost Allocation

---

## 🟢 What It Is in Simple Terms

Tag policies enforce consistent tagging standards across all accounts in an organization. Cost allocation tags tell AWS Cost Explorer to break down costs by specific tag values. Together they answer the critical question every finance and engineering leader asks: "How much are we spending on each team, product, environment, and service?"

---

## 🔍 Why Tags Matter Operationally

```
Without tags (chaos):
AWS bill: $500,000/month
Finance: "Where is this money going?"
Engineering: "Which team owns this EC2 instance?"
Incident: "Who owns this resource? Who do we page?"
Budget: "Is the payments team over budget this month?"
→ No answers. Manual investigation. Hours wasted.

With consistent tags (visibility):
aws ce get-cost-and-usage \
  --time-period Start=2024-01-01,End=2024-02-01 \
  --granularity MONTHLY \
  --group-by '[{"Type":"TAG","Key":"Team"}]'
→ Finance: payments=$120K, platform=$85K, data=$75K, etc.
→ Incident: check Team tag → find owner immediately
→ Budget: set CloudWatch alarm per Team tag cost threshold
```

---

## 🧩 Tag Policies (AWS Organizations)

```
Tag Policy = enforces tag naming conventions and allowed values
             Applied at org, OU, or account level (same as SCPs)

What tag policies enforce:
├── Tag key case consistency: "Environment" not "environment" or "ENV"
```

```
├── Required tag keys on specific resource types
├── Allowed values: Environment must be prod/staging/dev (not "prd",
"PROD")
└── Cannot add more restrictions than parent (policies are cumulative)

Tag policy example:
{
  "tags": {
    "Environment": {
      "tag_key": {
        "@@assign": "Environment"                ← exact case required
      },
      "tag_value": {
        "@@assign": ["prod", "staging", "dev", "sandbox"]  ← allowed
values only
      },
      "enforced_for": {
        "@@assign": [
          "ec2:instance",
          "ec2:volume",
          "rds:db",
          "s3:bucket",
          "lambda:function"
        ]
      }
    },
    "Team": {
      "tag_key": {"@@assign": "Team"},
      "tag_value": {
        "@@assign": ["payments", "platform", "data", "security",
"frontend"]
      }
    },
    "CostCenter": {
      "tag_key": {"@@assign": "CostCenter"}
      // No value restriction — any value allowed, but key must exist
    }
  }
}
```

```
# Create tag policy
aws organizations create-policy \
  --name "org-tagging-standards" \
  --type TAG_POLICY \
  --description "Enforce consistent tagging across all accounts" \
  --content file://tag-policy.json

# Attach to root OU (applies to all accounts)
aws organizations attach-policy \
  --policy-id p-tagging123 \
  --target-id r-root123
```

```
# Check tag compliance across organization
aws organizations get-resources-with-tag-policy-violations \
  --tag-policy-id p-tagging123
# Returns: which resources in which accounts have non-compliant tags

# Alternative: use Resource Groups Tagging API for bulk compliance check
aws resourcegroupstaggingapi get-resources \
  --tag-filters Key=Environment,Values=PROD  # find wrong-case "PROD" tags
aws resourcegroupstaggingapi get-tag-keys    # list all tag keys in use
```

---

## 🧩 Cost Allocation Tags

```
Cost Allocation Tags = tag keys activated in Billing console
                       so they appear as columns in Cost Explorer

Two types:
├── AWS-generated:  cost allocation tags AWS creates (e.g., aws:createdBy)
└── User-defined:   your tags (Environment, Team, Product, CostCenter)

Activation process:
1. Tag resources with your keys (Environment, Team, etc.)
2. Go to: Billing Console → Cost Allocation Tags → activate your tag keys
3. Wait 24 hours for Cost Explorer to start tracking by these tags
4. Query: cost breakdown by Team, Environment, Product

⚠ Activate tags BEFORE you need historical data.
   Cost allocation tags only track costs AFTER activation.
   Retroactive tagging doesn't retroactively apply to billing data.

Cost Explorer with tag breakdown:
aws ce get-cost-and-usage \
  --time-period Start=2024-01-01,End=2024-02-01 \
  --granularity MONTHLY \
  --metrics BlendedCost UnblendedCost \
  --group-by '[{"Type":"TAG","Key":"Team"},
{"Type":"DIMENSION","Key":"SERVICE"}]'

# Output:
# Team=payments: EC2=$45K, RDS=$30K, Lambda=$8K → $83K total
# Team=platform: EC2=$20K, EKS=$35K, VPN=$5K   → $60K total
# Team=data:     S3=$15K, Glue=$25K, Athena=$8K → $48K total

Untagged resources:
# Resources with no Team tag appear under: Team="" (empty string)
# These are orphaned/unowned resources → target for cleanup
```

---

## 🧩 Cost Allocation Strategy

```
Recommended tagging taxonomy:
├── Environment:  prod / staging / dev / sandbox
├── Team:         payments / platform / data / security
├── Product:      checkout / search / recommendations / auth
├── CostCenter:   engineering / infrastructure / security / r&d
├── ManagedBy:    terraform / cdk / cloudformation / manual
└── Owner:        alice@company.com (for automated resource cleanup)

Enforcing tags with Config rules:
# Non-compliant if EC2 instance missing required tags
aws config put-config-rule \
  --config-rule '{
    "ConfigRuleName": "required-tags-ec2",
    "Source": {
      "Owner": "AWS",
      "SourceIdentifier": "REQUIRED_TAGS"
    },
    "InputParameters": "{
      \"tag1Key\": \"Environment\",
      \"tag2Key\": \"Team\",
      \"tag3Key\": \"CostCenter\"
    }",
    "Scope": {
      "ComplianceResourceTypes": ["AWS::EC2::Instance"]
    }
  }'

Enforcing tags with SCP (prevent resource creation without required tags):
{
  "Effect": "Deny",
  "Action": "ec2:RunInstances",
  "Resource": "arn:aws:ec2:*:*:instance/*",
  "Condition": {
    "Null": {
      "aws:RequestTag/Environment": "true",
      "aws:RequestTag/Team":        "true"
    }
  }
}
# Cannot launch EC2 instance without providing BOTH Environment and Team
tags

Tagging at scale with Tag Editor:
# Find all untagged resources across region
aws resourcegroupstaggingapi get-resources \
  --tag-filters Key=Environment  # returns resources WITHOUT this tag if
empty
  --resource-type-filters "ec2:instance" "rds:db" "s3:bucket"

# Bulk tag multiple resources at once
aws resourcegroupstaggingapi tag-resources \
  --resource-arn-list arn:aws:ec2:...:instance/i-abc \
```

```
                        arn:aws:ec2:...:instance/i-def \
  --tags Environment=prod,Team=platform

Terraform default_tags (tag all resources consistently):
provider "aws" {
  default_tags {
    tags = {
      Environment = var.environment
      Team        = var.team
      ManagedBy   = "Terraform"
      Repository  = var.repository_url
    }
  }
}
# Every resource created by this provider gets these tags automatically
# No need to add tags to each individual resource block
```

---

## 🧩 AWS Budgets and Cost Alerts

```
Budgets with tag filters:
# Create budget: alert if Team=payments exceeds $100K/month
aws budgets create-budget \
  --account-id 123456789012 \
  --budget '{
    "BudgetName":   "payments-team-monthly",
    "BudgetType":   "COST",
    "TimeUnit":     "MONTHLY",
    "BudgetLimit":  {"Amount": "100000", "Unit": "USD"},
    "CostFilters": {
      "TagKeyValue": ["user:Team$payments"]
    },
    "CostTypes": {
      "IncludeTax": true,
      "IncludeSupport": false
    }
  }' \
  --notifications-with-subscribers '[{
    "Notification": {
      "NotificationType":  "ACTUAL",
      "ComparisonOperator": "GREATER_THAN",
      "Threshold": 80
    },
    "Subscribers": [{
      "SubscriptionType": "EMAIL",
      "Address":          "payments-team@company.com"
    }]
  }]'
# Alert at 80% of $100K = when payments team has spent $80K

Chargeback and showback:
```

```
  ├── Showback: report cost per team (informational — no financial transfer)
  └── Chargeback: allocate actual AWS costs to team P&Ls
      Use Cost Explorer CSV exports → finance team processes chargebacks
      Or: use AWS Cost and Usage Report (CUR) for detailed billing data
```

---

## 💬 Short Crisp Interview Answer

*"Tag policies in AWS Organizations enforce consistent tag key naming and allowed values across all accounts — so Environment must be 'prod' not 'PROD' or 'prd', and Team must be from an approved list. Cost allocation tags activate specific tag keys in the Billing console so Cost Explorer can break down costs by team, product, environment, or cost center — critical for financial accountability in multi-team environments. The activation lag is the key gotcha: tags only track billing data after activation, not retroactively. Enforce tags proactively: SCP denies resource creation without required tags, Config REQUIRED_TAGS rule reports non-compliant resources, and Terraform default_tags automatically applies standard tags to every resource. AWS Budgets with tag filters sends alerts when a specific team's spend exceeds threshold."*

---

## ⚠ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| Tag activation not retroactive | Must activate cost allocation tags BEFORE you need historical data |
| 24-hour delay | After activation, Cost Explorer takes ~24 hours to reflect new tag dimensions |
| Tag policy doesn't prevent | Tag policy reports violations but doesn't block resource creation (unlike SCP) |
| Case sensitivity matters | "Environment" and "environment" are different tag keys. Enforce case with tag policy |
| Not all resources support all tags | Some older AWS services don't support tagging — check documentation per service |
| Untagged resources cost allocation | Costs from untagged resources appear as empty string — creates "unknown" cost bucket |
| Terraform default_tags vs resource tags | Resource-level tags override default_tags for the same key — be aware of priority |

---

# 🔗 Category 10 — Full Connections Map

```
INFRASTRUCTURE & AUTOMATION connects to:

SSM
```

```
├── EC2                  → SSM Agent on instances enables all SSM features
├── IAM                  → instance role needs AmazonSSMManagedInstanceCore
├── KMS                  → Parameter Store SecureString encrypted with KMS
├── S3                   → session logs, Run Command output, Patch logs
├── CloudWatch Logs      → session streaming, Run Command output
├── CloudTrail           → all SSM actions (session start, Run Command)
logged
├── EventBridge          → SSM events (patch compliance, Run Command
status)
├── Config               → SSM Config rule for patch compliance
└── VPC Endpoints        → required for SSM in private subnets (3
endpoints)

Terraform
├── S3                   → remote state storage (versioned, encrypted)
├── DynamoDB             → state locking (LockID key per state file)
├── IAM                  → Terraform deployer role (OIDC from CI/CD)
├── All AWS services     → creates/manages any AWS resource via provider
└── CloudFormation       → CDK compiles to CF; Terraform uses AWS API
directly

CDK
├── CloudFormation       → CDK synthesizes to CF templates (deployment
engine)
├── S3                   → CDK bootstrap bucket for Lambda zips + CF
templates
├── ECR                  → CDK bootstrap repo for Docker image assets
├── IAM                  → CDK bootstrap roles (deploy, lookup, publishing)
└── All AWS services     → L1/L2/L3 constructs map to all AWS services

SCPs
├── AWS Organizations    → SCPs live in Organizations, attached to
OUs/accounts
├── IAM                  → SCPs restrict what IAM policies can grant
├── All AWS services     → SCPs can restrict any AWS service action
└── Control Tower        → CT uses SCPs as preventive guardrails

Control Tower
├── Organizations        → creates OU structure, enrolls accounts
├── IAM Identity Center  → SSO for all enrolled accounts
├── CloudTrail           → org trail to Log Archive account (mandatory
guardrail)
├── Config               → enabled in all accounts (mandatory guardrail)
├── S3                   → Log Archive account bucket for all logs
├── SNS                  → compliance alerts to Audit account
├── SCPs                 → preventive guardrails implemented as SCPs
└── Config Rules         → detective guardrails implemented as Config rules

RAM
├── Organizations        → share with org/OU/specific accounts
├── EC2                  → share Transit Gateway, subnets, capacity
reservations
├── Route53              → share Resolver rules, DNS Firewall rule groups
├── VPC                  → subnet sharing for centralized VPC management
```

```
  └── License Manager     → share license configurations across accounts

  Tag Policies
  ├── Organizations       → tag policies attached to OUs/accounts like SCPs
  ├── Cost Explorer       → cost allocation tags break down costs by tag
  ├── AWS Budgets         → budget alerts filtered by tag key-value
  ├── Config              → REQUIRED_TAGS rule detects missing tags
  ├── Resource Groups     → group resources by tag for management
  └── SCPs                → SCP denies resource creation without required
  tags
```

---

## 📌 Quick Reference — Interview Cheat Sheet

| Topic | Key Fact |
|---|---|
| SSM Agent connectivity | Communicates OUTBOUND only on HTTPS 443 — no inbound ports needed |
| SSM in private subnets | Needs 3 VPC endpoints: ssm, ssmmessages, ec2messages |
| SSM minimum IAM policy | AmazonSSMManagedInstanceCore attached to EC2 instance role |
| Session Manager logging | Not enabled by default — must configure S3 + CloudWatch logging in Session preferences |
| Port forwarding via SSM | aws ssm start-session with AWS-StartPortForwardingSession document — no SG inbound rule needed |
| Run Command rate control | max-concurrency (e.g., "25%") + max-errors (e.g., "10%") for safe rolling execution |
| Patch Manager reboot | Does NOT auto-reboot — set RebootOption=RebootIfNeeded explicitly |
| Patch Group tag | Tag EC2 with "Patch Group" = "prod-web" → baseline applies to that group |
| State Manager vs Run Command | State Manager = ongoing/scheduled enforcement. Run Command = one-time execution |
| State Manager drift | Re-applies configuration at schedule — manual changes get overwritten |
| Terraform remote state | S3 (versioned + encrypted) for state file + DynamoDB for lock |
| DynamoDB lock item | LockID = full S3 state path. Created at plan start, deleted after apply |
| terraform force-unlock | Use when apply crashes and lock item persists in DynamoDB |
| State file sensitivity | Contains plaintext secrets — encrypt S3 bucket with KMS |
| Terraform OIDC | GitHub Actions assumes AWS role via OIDC — no stored access keys |
| CDK L1 constructs | Cfn* classes — 1:1 to CloudFormation, all properties required explicitly |
| CDK L2 constructs | Opinionated defaults, expressive APIs (grantRead, addLifecycleRule) |
| CDK L3 constructs | Complete patterns (ApplicationLoadBalancedFargateService) |

| Topic | Key Fact |
|---|---|
| CDK bootstrap | Runs once per account+region — creates S3 bucket, ECR repo, IAM roles |
| CDK bootstrap stack | CloudFormation stack named CDKToolkit |
| cdk synth makes AWS calls | Context lookups need AWS credentials even for synth — not just deploy |
| SCP evaluation | SCPs restrict ALL principals including root. Cannot be overridden by IAM allow |
| SCP on management account | SCPs don't apply to the organization management account |
| SCP NotAction for global services | Region-restriction SCP must exclude IAM, Route53, CloudFront via NotAction |
| Break-glass exception | Exclude SecurityAdminRole from SCP deny conditions for emergency access |
| Control Tower setup time | Initial landing zone takes 60-90 minutes to deploy |
| Control Tower accounts | Creates: Management + Log Archive + Audit accounts automatically |
| Mandatory guardrails | Cannot be disabled — protect CloudTrail, Config, log archive account |
| Control Tower drift | Someone changes CT-managed config externally → shows as DRIFTED |
| Drift repair | aws controltower reset-landing-zone restores expected state |
| Account Factory | Provisions new accounts in ~30 minutes with all guardrails auto-applied |
| RAM sharing with org | Share with org ARN → new accounts automatically receive shared resources |
| RAM Transit Gateway sharing | One TGW in network account, shared to 200 accounts — no per-account cost |
| RAM subnet sharing | Workload accounts launch resources into shared subnets — IP from central VPC |
| RAM Security Groups | Security Groups CANNOT be shared via RAM — each account creates own |
| RAM resource cost billing | Costs bill to owner account, not to accounts using the shared resource |
| Tag policy enforcement | Reports violations but does NOT block creation (unlike SCP) |
| Cost allocation tag activation | 24-hour delay after activation. No retroactive billing data |
| SCP tag enforcement | Deny resource creation when required tags absent using aws:RequestTag/Key Null condition |
| Terraform default_tags | Apply standard tags to all resources via provider default_tags block |

| Topic | Key Fact |
|---|---|
| Untagged cost allocation | Resources with no tag appear as empty string in Cost Explorer |

*Category 10: Infrastructure & Automation — Complete Interview Guide*
*Excluded topics: CodeBuild, CodePipeline, CloudFormation*